

INFORME TÉCNICO

Sistema de Gestión y Optimización de Rutas de Emergencia

Nombre: José Villablanca Alarcón / ICC708-1

1.1 Contexto y Problemática

La gestión eficiente de emergencias urbanas depende críticamente de dos factores: la capacidad de priorizar correctamente los incidentes según su gravedad y la capacidad de calcular, en tiempo real, la ruta más rápida posible desde un centro de atención disponible hasta el lugar del suceso. En contextos de alta densidad poblacional, los sistemas de despacho tradicionales basados en listas simples o en la intervención manual de un operador tienden a degradarse rápidamente cuando el volumen de incidentes crece, produciendo tiempos de respuesta subóptimos y, en el peor de los casos, la postergación indebida de casos críticos frente a otros de menor severidad que llegaron antes.

Este problema no es solamente operativo, sino también algorítmico: exige estructuras de datos capaces de sostener operaciones de inserción y extracción de máximos en tiempo logarítmico, mecanismos de búsqueda con acceso casi constante para el seguimiento de cada caso, y algoritmos de búsqueda de caminos que consideren el costo real de desplazamiento y no solamente el número de tramos recorridos para garantizar que el recurso de emergencia asignado sea efectivamente el más rápido y no simplemente el más cercano en número de intersecciones. El presente informe documenta el diseño, la implementación y la validación experimental de un prototipo académico que aborda estos desafíos combinando tablas hash, colas de prioridad basadas en montículos binarios, grafos dispersos y algoritmos clásicos de búsqueda y ordenamiento.

1.2 Diseño del Sistema

El sistema se organiza en módulos independientes, cada uno responsable de una capa concreta de la solución. Esta separación de responsabilidades facilita tanto las pruebas unitarias como el mantenimiento futuro del código, y permite razonar sobre la complejidad computacional de cada componente de manera aislada.

- **models:** Define los tipos de datos abstractos centrales del dominio: Incident, EmergencyCenter y RoadNetwork, encapsulando sus atributos y comportamientos propios.
- **hash_table:** Implementa una tabla hash propia con resolución de colisiones por encadenamiento, utilizada para indexar y recuperar incidentes en tiempo esperado $O(1)$.
- **priority_queue:** Implementa una cola de prioridad basada en un Max-Heap, empleada para extraer siempre el incidente de mayor urgencia en tiempo $O(\log n)$.
- **Sorting:** Contiene las implementaciones de MergeSort y QuickSort, utilizadas para ordenar reportes de incidentes bajo distintos criterios (prioridad, marca temporal, entre otros).
- **search_algorithms:** Implementa Búsqueda en Anchura (BFS) para minimizar el número de tramos recorridos, y Búsqueda de Costo Uniforme / Dijkstra para minimizar el tiempo real de viaje sobre el grafo vial.

- **main:** Actúa como punto de entrada del prototipo, proveyendo una interfaz de línea de comandos interactiva para simular el ciclo completo de atención de incidentes.
- **test_suite:** Conjunto de pruebas de validación automatizadas que verifica la correctitud funcional de cada módulo, con un resultado consolidado de 70 de 70 pruebas superadas.

La siguiente tabla resume la responsabilidad principal de cada módulo, las estructuras de datos clave que utiliza y su complejidad temporal dominante:

Módulo	Responsabilidad principal	Estructura clave	Complejidad dominante
models	Definición de ADTs del dominio	Clases Incident, EmergencyCenter, RoadNetwork	$O(1)$ por operación de atributo
hash_table	Indexación y búsqueda de incidentes	Arreglo de buckets encadenados	$O(1)$ promedio / $O(n)$ peor caso
priority_queue	Extracción del incidente más urgente	Max-Heap sobre arreglo + índice de posiciones	$O(\log n)$ inserción/extracción
sorting	Ordenamiento de reportes	MergeSort y QuickSort	$O(n \log n)$ promedio
search_algorithms	Cálculo de rutas	BFS y Dijkstra sobre lista de adyacencia	$O(V+E)$ y $O((V+E) \log V)$
main	Orquestación e interfaz CLI	Composición de los módulos anteriores	Depende del flujo invocado
test_suite	Validación funcional	Casos de prueba unitarios e integración	70/70 pruebas exitosas

2. Diseño de Estructuras de Datos (ADTs) y Complejidad Temporal

2.1 ADT Incident y Mecanismo de Priorización

El núcleo de la lógica de despacho reside en la fórmula que determina la prioridad efectiva de cada incidente. Dicha fórmula se define como: $\text{prioridad} = \text{severidad} \times \text{peso_tipo} \times (1.0 + \text{minutos_transcurridos} \times 0.05)$. El primer factor, *severidad*, representa la gravedad intrínseca reportada del incidente. El segundo factor, *peso_tipo*, es un coeficiente que ajusta la prioridad según la naturaleza del evento de forma que, por ejemplo, un incendio estructural pueda ponderarse de manera distinta a un accidente de tránsito menor. El tercer factor introduce el concepto de envejecimiento o *aging*, mediante el cual la prioridad de un incidente aumenta de forma proporcional al tiempo transcurrido desde su reporte.

El propósito del término de *aging* es evitar la inanición de incidentes de menor severidad relativa: sin este mecanismo, un incidente de severidad baja podría permanecer indefinidamente relegado en la cola si continúan llegando incidentes de severidad más alta. Al incrementar la prioridad en un 5% por cada minuto transcurrido, el sistema garantiza que, tarde o temprano, todo incidente alcance una prioridad suficientemente alta como para ser atendido, incluso si su severidad original era modesta. Este comportamiento introduce una noción de equidad temporal dentro de un esquema que, de otro modo, sería puramente jerárquico por gravedad.

2.2 ADT HashTable (Tabla Hash)

La tabla hash implementada resuelve colisiones mediante encadenamiento: cada posición del arreglo interno almacena una lista de tuplas clave-valor, de modo que múltiples claves que colisionan en el mismo índice conviven en la misma lista sin sobrescribirse. La función hash empleada es de tipo polinomial, que combina los caracteres o componentes de la clave mediante una combinación ponderada, lo cual favorece una distribución razonablemente uniforme de las claves sobre el espacio de buckets disponibles y reduce la probabilidad de agrupamientos.

Para mantener el rendimiento a medida que la tabla crece, se implementa un mecanismo de redimensionamiento automático: cuando el factor de carga definido como la razón entre el número de elementos almacenados y la capacidad total de buckets supera el umbral de 0.75, la tabla duplica su capacidad interna y reinserta todos los elementos existentes en la nueva estructura. Este proceso, aunque costoso en el momento puntual en que ocurre ($O(n)$), se amortiza sobre el conjunto de todas las inserciones, de modo que el costo promedio por operación se mantiene en $O(1)$.

Operación	Mejor caso	Caso promedio	Peor caso
Inserción	$O(1)$	$O(1)$	$O(n)$
Búsqueda	$O(1)$	$O(1)$	$O(n)$
Eliminación	$O(1)$	$O(1)$	$O(n)$

El peor caso $O(n)$ ocurre únicamente cuando la función hash degenera y todas las claves colisionan en un mismo bucket, situación que la política de redimensionamiento automático busca precisamente evitar, manteniendo el tamaño de los buckets acotado en la práctica.

2.3 ADT Max-Heap (Cola de Prioridad)

El Max-Heap se representa internamente como un arreglo lineal, aprovechando las propiedades aritméticas de un árbol binario completo: para un nodo ubicado en la posición i , su padre se encuentra en la posición $\lfloor (i-1)/2 \rfloor$, su hijo izquierdo en $2i+1$ y su hijo derecho en $2i+2$. Esta representación evita el uso de punteros explícitos entre nodos y permite un acceso directo y eficiente en memoria contigua, lo que resulta especialmente favorable en términos de localidad de caché.

Una de las decisiones de diseño más relevantes es la incorporación de un índice de posiciones auxiliar (`_pos_index`), que mantiene, para cada incidente, la posición exacta que ocupa dentro del arreglo del heap. Sin esta estructura auxiliar, actualizar la prioridad de un incidente ya presente en la cola requeriría recorrer linealmente el arreglo para localizarlo, con un costo $O(n)$. Gracias al índice de posiciones, la operación `update_priority` puede localizar el elemento en $O(1)$ y luego aplicar las operaciones de sift-up o sift-down necesarias en $O(\log n)$, evitando así el cuello de botella que representaría una búsqueda lineal repetida en un sistema donde las prioridades cambian constantemente por efecto del aging.

La construcción inicial del heap a partir de un conjunto de elementos no ordenados se realiza mediante el algoritmo de Floyd (heapify ascendente desde el último nodo interno hacia la raíz), cuyo costo total es $O(n)$ y no $O(n \log n)$ como podría suponerse ingenuamente, dado que la mayoría de los nodos se encuentran cerca de las hojas y requieren pocas operaciones de hundimiento.

2.4 ADT RoadNetwork (Grafo)

La red vial se modela mediante listas de adyacencia manuales en lugar de una matriz de adyacencia. Esta decisión responde directamente a la naturaleza dispersa (sparse) de las redes viales reales: el número de conexiones directas entre intersecciones es, en la práctica, mucho menor que el máximo teórico de n^2 conexiones posibles entre n nodos. Mientras que una matriz de adyacencia consumiría $O(n^2)$ de memoria independientemente de cuántas conexiones existan realmente, una lista de adyacencia consume $O(V+E)$, proporcional al número real de vértices y aristas, lo cual resulta considerablemente más eficiente en memoria para grafos viales de tamaño urbano. Adicionalmente, recorrer los vecinos de un nodo operación central en BFS y Dijkstra es directa en una lista de adyacencia, mientras que en una matriz implicaría recorrer una fila completa de tamaño n , incluyendo entradas nulas.

3. Diagramas y Flujo Operacional (Diseño Conceptual)

3.1 Relación de Clases

En memoria, los distintos ADTs se relacionan de forma cooperativa: cada objeto Incident es referenciado simultáneamente desde dos estructuras independientes la HashTable, que lo indexa por su identificador único para acceso directo, y el Max-Heap, que lo posiciona según su prioridad calculada. Esta dualidad de referencias es posible porque ambas estructuras almacenan referencias al mismo objeto en memoria (o a una clave común), de modo que una actualización del estado de un incidente por ejemplo, marcarlo como 'en_atencion' es visible de manera consistente desde ambas vías de acceso. El objeto RoadNetwork, por su parte, opera de forma independiente como un grafo de soporte, consultado únicamente en el momento de calcular la ruta hacia el incidente ya seleccionado, y los objetos EmergencyCenter representan los nodos origen candidatos dentro de ese mismo grafo.

3.2 Flujo Operacional del Escenario

El ciclo completo de atención de una emergencia sigue una secuencia de tres etapas encadenadas:

- 1. Extracción del incidente más urgente: se invoca la operación de extracción de máximo sobre el Max-Heap, obteniendo en $O(\log n)$ el incidente con la prioridad efectiva más alta en ese instante, considerando tanto su severidad original como el efecto acumulado del aging.
- 2. Localización y actualización de estado: el incidente extraído se busca en la HashTable mediante su identificador, en tiempo esperado $O(1)$, para actualizar su estado a 'en_atencion' y evitar que sea procesado nuevamente por otra unidad.
- 3. Cálculo de la ruta óptima: se ejecuta el algoritmo de Dijkstra sobre el RoadNetwork, partiendo de todos los centros de emergencia disponibles hacia la ubicación del incidente, con el fin de determinar tanto el centro más cercano en tiempo real de viaje como la ruta específica que debe recorrer la unidad despachada.

Esta secuencia garantiza que, en cada iteración del ciclo de despacho, el sistema atiende siempre al incidente objetivamente más urgente disponible en ese momento, y que la unidad asignada corresponda a la que ofrece el menor tiempo de llegada real, y no simplemente a la geográficamente más próxima en línea recta o en número de intersecciones.

4. Análisis de Algoritmos de Búsqueda y Ordenamiento

4.1 BFS vs. Dijkstra

La Búsqueda en Anchura (BFS) explora el grafo nivel por nivel, garantizando encontrar el camino con el menor número de aristas (saltos) entre un origen y un destino. Sin embargo, esta garantía es válida únicamente cuando todas las aristas tienen el mismo costo o peso. En una red vial real, cada tramo tiene un tiempo de recorrido distinto, determinado por factores como la distancia, el tipo de vía o la congestión esperada. En este contexto, BFS puede seleccionar una ruta con menos intersecciones pero con tramos individualmente más lentos, resultando en un tiempo total de viaje mayor al de una alternativa con más tramos pero de menor duración conjunta.

El algoritmo de Dijkstra, en cambio, opera sobre el principio de costo acumulado: en cada paso explora el nodo no visitado con menor costo total desde el origen, garantizando siempre que no existan pesos negativos, condición que se cumple en tiempos de viaje que la primera vez que un nodo se marca como visitado, su costo acumulado es efectivamente el mínimo posible. Por esta razón, Dijkstra es el algoritmo adecuado para minimizar el tiempo real de viaje en lugar del número de tramos recorridos, a costa de una complejidad ligeramente mayor, $O((V+E) \log V)$ frente a los $O(V+E)$ de BFS.

4.2 MergeSort vs. QuickSort

MergeSort es un algoritmo de ordenamiento por división y conquista con complejidad garantizada de $O(n \log n)$ en todos los casos (mejor, promedio y peor), a costa de un uso adicional de memoria $O(n)$ para las fusiones. QuickSort, por su parte, alcanza también $O(n \log n)$ en el caso promedio, pero puede degradar a $O(n^2)$ en el peor caso, particularmente cuando la selección del pivote es desfavorable frente a datos ya ordenados o con muchos valores repetidos; en contrapartida, QuickSort suele ser más rápido en la práctica gracias a un menor factor constante y a su naturaleza in-place, que reduce el uso de memoria auxiliar.

Un aspecto crucial para este dominio de aplicación es la estabilidad del algoritmo de ordenamiento, entendida como la propiedad de preservar el orden relativo original entre elementos que comparten la misma clave de ordenamiento. MergeSort es estable por construcción, mientras que QuickSort, en sus implementaciones estándar, no lo es. Esta diferencia resulta relevante cuando dos o más incidentes comparten exactamente la misma prioridad: un algoritmo estable preserva el orden de llegada entre ellos, atendiendo primero al que fue reportado antes, lo cual resulta más justo y predecible desde una perspectiva operativa. Por esta razón, MergeSort es preferible cuando se requiere mantener el orden de llegada como criterio de desempate implícito.

5. Pruebas Experimentales y Resultados Reales

A continuación se presenta el análisis de los resultados obtenidos mediante el conjunto de pruebas de benchmark ejecutado sobre el prototipo, correspondientes al rendimiento del Max-Heap, la comparación entre algoritmos de ordenamiento, la comparación entre BFS y Dijkstra, y las métricas internas de la tabla hash.

5.1 Rendimiento del Max-Heap (Inserción + Extracción)

N° de elementos	T_insert (ms)	T_extract (ms)
100	1.0287	2.5447
200	3.4036	11.6818
300	7.8162	27.5192
400	15.9883	47.5672
500	23.7440	76.2919

Los tiempos medidos para la inserción y la extracción crecen de manera consistente con el tamaño de la entrada, pero no de forma estrictamente lineal: al pasar de 100 a 500 elementos (un incremento de cinco veces), el tiempo de inserción aumenta poco más de veintitrés veces y el de extracción alrededor de treinta veces, un crecimiento superlineal esperable en una estructura $O(\log n)$ por operación individual cuando se acumulan n operaciones consecutivas (es decir, un comportamiento agregado cercano a $O(n \log n)$ para el conjunto completo de inserciones o extracciones). El crecimiento notablemente más pronunciado de los tiempos de extracción respecto a los de inserción es coherente con el costo adicional que implica el proceso de sift-down tras remover la raíz del heap, el cual, en promedio, recorre una porción mayor del árbol que el sift-up típico de una inserción.

En conjunto, estos resultados validan empíricamente el comportamiento logarítmico esperado de la estructura: si el crecimiento fuera lineal por operación ($O(n)$ en lugar de $O(\log n)$), el incremento de tiempos al quintuplicar el tamaño de la entrada habría sido cercano a veinticinco veces de forma estable en ambas operaciones, mientras que las diferencias observadas y su moderada divergencia entre inserción y extracción reflejan con mayor fidelidad la naturaleza logarítmica por operación combinada con el efecto acumulativo de realizar n operaciones sucesivas sobre la estructura durante el benchmark.

5.2 Comparativa de Ordenamiento ($n = 500$)

Criterio de ordenamiento	MergeSort (ms)	QuickSort (ms)
Prioridad descendente	1.8277	3.6695
Timestamp ascendente	2.9673	1.5084

Los resultados muestran un comportamiento inverso según el criterio de ordenamiento aplicado. Al ordenar por prioridad descendente, MergeSort resulta aproximadamente dos veces más rápido que QuickSort, lo cual sugiere que la distribución de las prioridades posiblemente con valores repetidos o parcialmente agrupados como consecuencia del efecto de aging aplicado de forma homogénea generó particiones desbalanceadas en QuickSort, acercando su comportamiento al peor caso cuadrático y penalizando su desempeño frente a la garantía estable de MergeSort.

En cambio, al ordenar por marca temporal (timestamp) ascendente, QuickSort resulta más rápido que MergeSort. Esto es consistente con una distribución de timestamps más uniforme y menos propensa a generar pivotes desfavorables, permitiendo que QuickSort explote su menor factor constante y su naturaleza in-place, sin el costo de memoria adicional que MergeSort necesariamente incurre en cada fusión. Este contraste entre ambos escenarios ilustra un principio general en la selección de algoritmos de ordenamiento: la elección óptima depende no solo de la complejidad asintótica teórica, sino también de las características concretas de la distribución de los datos de entrada y de la estrategia de selección de pivote empleada.

5.3 BFS vs. Dijkstra (Costo de ruta y nodos explorados)

Par	BFS Costo (min)	BFS Nodos visitados	Dijkstra Costo (min)	Dijkstra Nodos visitados
1	75.5	44	39.1	26
2	93.9	50	61.6	55
3	71.1	38	54.3	25
4	59.2	45	38.3	39
5	40.3	12	40.3	32

En cuatro de los cinco pares evaluados, Dijkstra encuentra una ruta con un costo temporal considerablemente menor al de BFS, con reducciones que oscilan entre aproximadamente el 8% (par 4) y el 48% (par 1) respecto al costo obtenido por BFS. Esto confirma de forma empírica la limitación teórica descrita en la sección 4.1: BFS minimiza el número de tramos recorridos, pero al ignorar el peso de cada arista, con frecuencia selecciona una ruta con menos intersecciones que, sumadas, representa mayor tiempo de viaje real que una ruta alternativa con más tramos pero individualmente más cortos en duración.

El caso del par 5 resulta particularmente ilustrativo: ambos algoritmos obtienen exactamente el mismo costo (40.3 minutos), lo que indica que, para ese origen y destino específicos, la ruta con menor número de tramos coincide con la ruta de menor tiempo total, un escenario posible cuando los pesos de las aristas en el camino más corto en saltos son homogéneos. Sin embargo, es interesante notar que Dijkstra exploró más nodos (32 frente a 12) para llegar a esa misma conclusión óptima, lo cual es coherente con su estrategia de expansión basada en costo acumulado, que en general requiere explorar un conjunto más amplio de nodos candidatos que una simple expansión por niveles, como parte del costo adicional que paga a cambio de la garantía de optimalidad en tiempo real de viaje.

5.4 Métricas de la Tabla Hash (n = 500)

Métrica	Valor
Capacidad (buckets)	1024
Elementos almacenados	500
Factor de carga	0.4883
Colisiones totales	622
Buckets en uso	213
Tamaño máximo de bucket	5 elementos
Rehashes realizados	4

Con 500 elementos distribuidos sobre 1024 buckets, el factor de carga observado (0.4883) se mantiene cómodamente por debajo del umbral de redimensionamiento de 0.75, lo cual explica por qué, a lo largo del proceso de inserción, la tabla requirió únicamente cuatro operaciones de rehashing para llegar a esta capacidad final desde un tamaño inicial menor. El hecho de que solamente 213 de los 1024 buckets estén en uso, con 622 colisiones totales, indica que existe una proporción de buckets que agrupan más de un elemento, lo cual es esperable incluso con una función hash bien distribuida cuando el número de elementos es significativamente menor que el número de buckets disponibles.

El dato más relevante desde el punto de vista de la complejidad práctica es que, a pesar de las 622 colisiones registradas, el bucket con mayor ocupación contiene únicamente 5 elementos. Esto significa que, en el peor caso observado experimentalmente, una operación de búsqueda o eliminación requeriría

recorrer, como máximo, una lista de 5 tuplas antes de encontrar la clave buscada —un costo que en la práctica es indistinguible de una operación en tiempo constante—. Este resultado valida empíricamente que la combinación de una función hash polinomial razonablemente uniforme junto con el mecanismo de rehashing automático logra mantener el acceso a la tabla efectivamente en $O(1)$, incluso bajo una carga considerable de elementos y colisiones acumuladas.

6. Conclusión

El prototipo desarrollado demuestra que la combinación deliberada de estructuras de datos clásicas —una tabla hash con encadenamiento y rehashing automático, una cola de prioridad basada en Max-Heap con índice de posiciones, un grafo representado mediante listas de adyacencia y algoritmos de búsqueda y ordenamiento seleccionados de acuerdo con las propiedades reales del dominio— constituye una solución técnicamente sólida para el problema de gestión y optimización de rutas de emergencia. Cada decisión de diseño, lejos de ser arbitraria, responde a una justificación de complejidad computacional concreta: el uso de listas de adyacencia frente a matrices para aprovechar la dispersión de la red vial, el índice de posiciones en el heap para evitar búsquedas lineales durante el aging, y la preferencia por MergeSort cuando la estabilidad del orden de llegada resulta operativamente relevante.

Los resultados experimentales respaldan de manera consistente el marco teórico expuesto: el crecimiento superlineal mas no lineal de los tiempos del Max-Heap confirma su naturaleza logarítmica por operación; la comparación entre BFS y Dijkstra evidencia de forma cuantitativa la superioridad de este último para minimizar tiempos reales de viaje en grafos con pesos heterogéneos; el contraste entre MergeSort y QuickSort bajo distintos criterios de ordenamiento ilustra que la elección algorítmica óptima depende también de la distribución específica de los datos; y las métricas de la tabla hash, con un tamaño máximo de bucket de apenas 5 elementos sobre 500 almacenados, confirman que el acceso indexado se mantiene efectivamente en tiempo constante en condiciones prácticas. Sumado a esto, la validación funcional completa, con 70 de 70 pruebas superadas, respalda la correctitud de la implementación en todos sus módulos.

En conjunto, estos elementos permiten concluir que el sistema es operativamente viable como prototipo de apoyo a la toma de decisiones en escenarios de despacho de emergencias, y que su arquitectura modular ofrece una base razonable para su escalabilidad futura. Ampliaciones naturales del trabajo incluirían la incorporación de pesos dinámicos que reflejen condiciones de tráfico en tiempo real, la paralelización del cálculo de rutas para múltiples incidentes simultáneos, y la persistencia de los datos en una base de datos externa que permita sostener volúmenes de incidentes muy superiores a los evaluados en el presente benchmark, sin comprometer las garantías de complejidad que este informe ha documentado y validado.